Forecast Agent

Hour-by-hour electricity demand forecasts for buildings, with a measure of how far to trust them

Project overview · MDM Hackathon · 17 October 2026

Forecast Agent predicts how much electricity a building will draw over the next 24 hours. It learns the building's daily and weekly rhythm and its response to weather from past meter readings. Two neural networks (an LSTM and an MLP) are trained, compared on a period they have never seen, and blended into a weighted ensemble. Every forecast comes with a typical error and an 80 % range, and is delivered as a report and in a web dashboard.

7.81 kW typical hourly error (test MAE), LSTM on a real office	**8.6 %** error relative to actual use (test WAPE)	**86 %** of actual test readings inside the 80 % range	**82** automated tests, all passing

Measured on the held-out test period 2017-09-13 to 2017-12-31 of a real 10,352 m² office building from the Building Data Genome Project 2.

1 What it is for

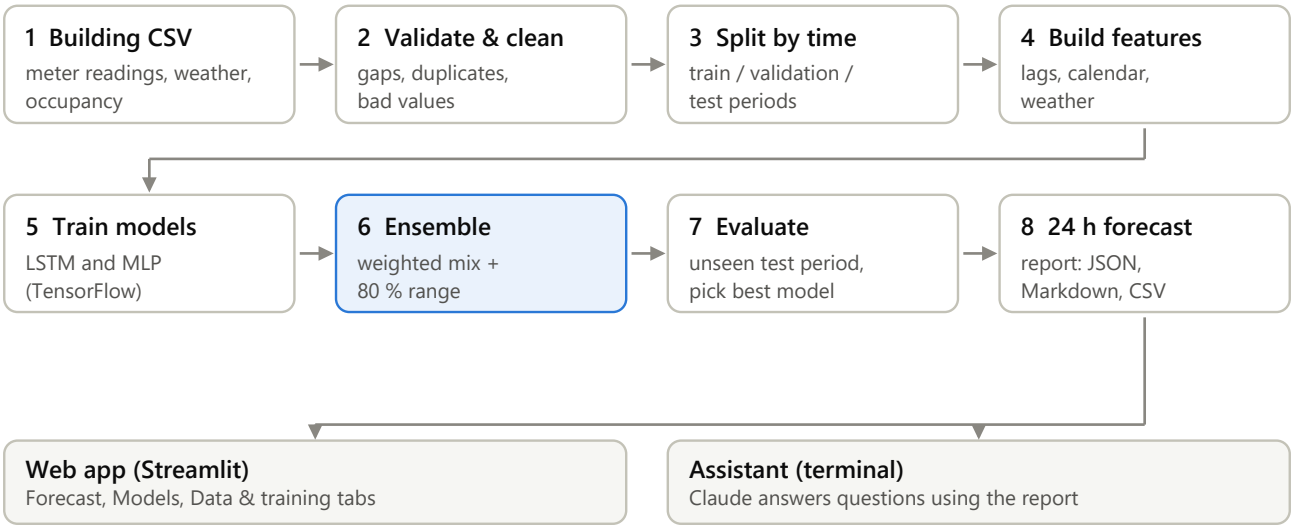
Electricity costs for commercial buildings depend not only on how much energy is used but on **when** it is used and on the **highest demand** reached. A reliable view of tomorrow's hourly demand lets a facility manager act before the cost is incurred.

Use	How the forecast helps
Avoid peak-demand charges	Many tariffs bill the month's highest kW. Knowing tomorrow's peak and its hour leaves time to shift loads, pre-cool or discharge a battery.
Schedule HVAC and equipment	Run heavy loads in the predicted low-demand hours.
Plan energy purchases	Buildings that buy power ahead need to know how much they will use.
Spot faults	Actual use far above the forecast points to equipment left on or a fault; the backtest mode compares the two directly.
Size solar and storage	Match on-site generation and batteries to the expected demand profile.

Scope. Forecast Agent forecasts; it does not act. It does not switch equipment, detect waste automatically or make recommendations. Its forecasts are only as good as the data it is trained on.

2 How it works

Every step is a plain Python module, so the command-line tools and the web app run exactly the same code. The ensemble stage (highlighted) is what links the two models.



- **1–2 Load and validate.** The CSV is checked for duplicate timestamps, gaps, irregular intervals, impossible values and too little data. Short gaps are filled forward only, never from the future.
- **3 Split by time.** The oldest 70 % trains the models, the next 15 % (validation) is used to choose and tune, and the newest 15 % (test) is kept unseen for honest scoring.
- **4 Features.** Demand 1, 2, 3, 24, 48 and 168 hours earlier; hour, weekday and month; a weekend flag; temperature, humidity, occupancy and equipment status when the data has them.
- **5 Train.** An LSTM reads the last 24 hours in sequence; an MLP uses the lagged values and features directly. Both predict all 24 hours at once and stop early when validation error stops improving.
- **6 Ensemble.** The forecasts are blended as w × LSTM + (1 − w) × MLP, with w fitted on the validation period. The 10th–90th percentile of validation errors gives an 80 % range for each hour ahead.
- **7 Evaluate.** All three are scored on the test period. The preferred model is picked on validation only.
- **8 Forecast.** The chosen model forecasts the 24 hours after the latest reading and writes one report in a fixed format, which the web app and the assistant both read.

3 Which part does what

Part	Files	What it does	Why it is built this way
Settings	`src/config.py`	Look-back and horizon (24 h), model sizes, split sizes, epochs, seed.	One place to change behaviour; every value can be overridden from the command line.
Data loading	`src/data_loader.py`	Reads the CSV, detects columns, parses timestamps, converts on/off values.	Real exports differ in date format, column case and status labels.
Validation and cleaning	`src/data_validation.py`	Reports gaps, duplicates, invalid readings and too-short history; cleans with forward-fill only.	Bad data silently ruins a model; stopping with a clear message is safer.

Part	Files	What it does	Why it is built this way
Splitting and scaling	`src/preprocessing.py` `src/pipeline.py`	Chronological train/validation/test split; scaler fitted on training rows only.	A time series cannot be shuffled: testing must use a later period.
Features	`src/feature_engineering.py`	Lagged demand, calendar encodings, weather and occupancy features.	Building load repeats daily and weekly and follows weather and people.
LSTM model	`src/models/lstm_model.py`	Recurrent network over the last 24 hours.	Learns patterns that build up over time.
MLP model	`src/models/mlp_model.py`	Feed-forward network over lagged values and features.	Fast, simple comparison model; often competitive.
Training	`src/train.py` `src/models/__init__.py`	Early stopping, fixed seed, saves model and metadata.	Prevents memorising the training data; runs are repeatable.
Saved models	`src/artifacts.py`	Stores model, scaler, settings, version and the training file's hash.	A model is only valid with its own preprocessing; the hash catches the wrong CSV.
Metrics	`src/metrics.py`	MAE, RMSE, WAPE, error by lead time, daily peak error.	Peak error is what drives demand charges, so it is measured on its own.
Evaluation	`src/evaluate.py`	Scores every model on the unseen test period; selects on validation; plots.	Choosing the winner on test data would make the scores meaningless.
Ensemble	`src/ensemble.py`	Fits the blend weight and 80 % range on validation; refuses to run if a model was retrained since.	Two models make different mistakes; the range shows uncertainty.
Forecasting	`src/predict.py` `src/inference.py`	24 h forecast after the latest reading, or a backtest from an earlier time.	Backtests compare the forecast with what actually happened.
Forecast report	`src/report.py`	One format for every model: summary, accuracy, range, hourly values, caveats; JSON and Markdown.	Dashboard, assistant and slides can all rely on the same structure.
Web app	`streamlit_app.py` `src/ui/charts.py`	Forecast, Models and Data & training tabs; charts, tables, downloads, one-click pipeline.	Usable without a terminal; one colour per model, colour-blind safe.
Assistant (RAG)	`src/rag/`	Searches documents (TF-IDF), adds the forecast report, asks Claude; answers in a fixed structure with sources.	Plain-language questions, answered only from the provided context.
Data tools	`tools/generate_dummy_dataset.py` `tools/convert_bdg2.py`	Synthetic demo building; converter for real buildings from the Building Data Genome Project 2.	Runs end to end on demo data, then on real data.
Tests	`tests/`	82 tests: validation, leakage, metrics, models, ensemble, report, assistant, UI, converter.	Shows the results come from correct code, not luck.

4 Results on a real office building

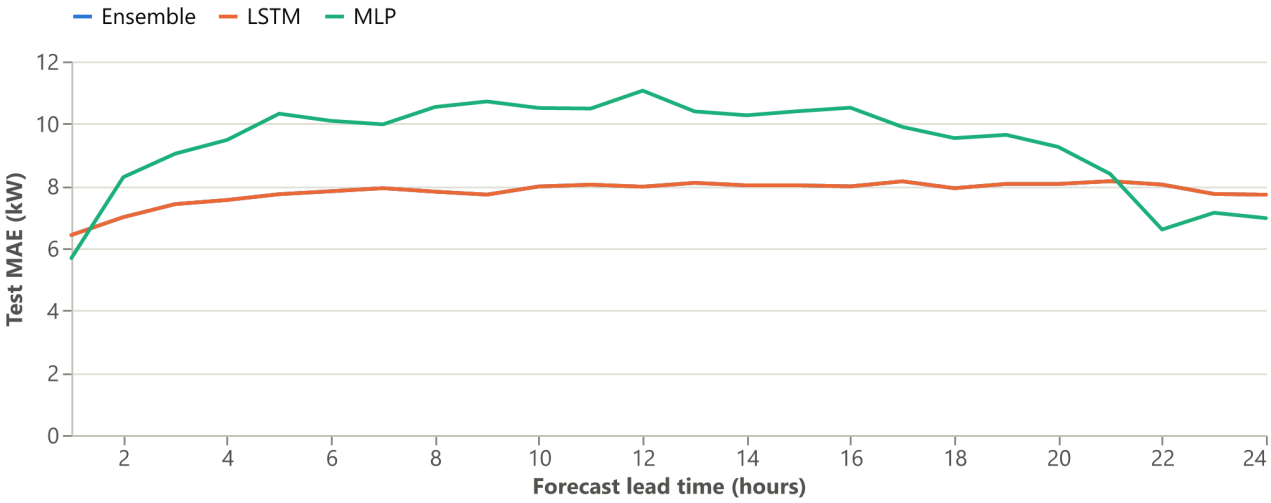
The models were trained on **Hog_office_Lizzie**, a 10,352 m² office in the US Central time zone from the Building Data Genome Project 2: 17,544 hourly readings from 2016-01-01 to 2017-12-31, with outdoor temperature and humidity from the site's weather station. This dataset has no occupancy or equipment-status data.

Period	Dates	Samples
Training	2016-01-01 to 2017-05-26	12,089
Validation (choose and tune)	2017-05-26 to 2017-09-13	2,609
Test (unseen, final scores)	2017-09-13 to 2017-12-31	2,609

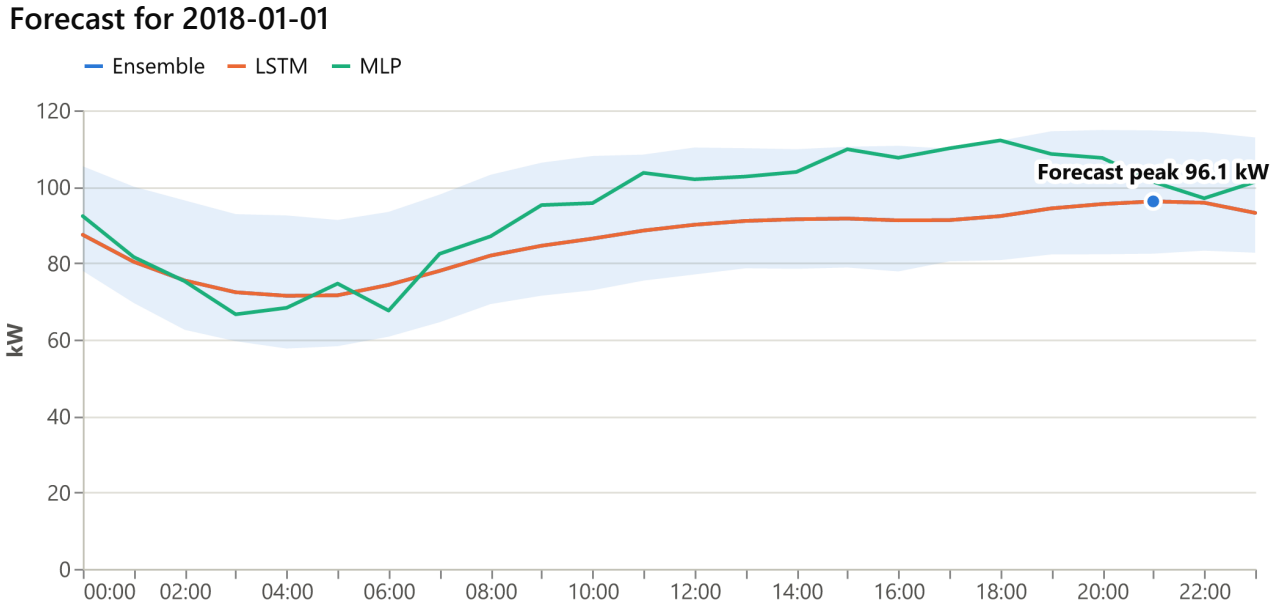
Model	Val MAE (kW)	Val RMSE (kW)	Test MAE (kW)	Test RMSE (kW)	Test WAPE	Daily peak error (kW)
LSTM (preferred)	9.65	13.69	7.81	10.53	8.6 %	6.92 (5.7 %)
MLP	13.17	18.86	9.38	12.31	10.3 %	10.16 (8.9 %)
Ensemble	9.65	13.69	7.81	10.53	8.6 %	6.92 (5.7 %)

- **The LSTM is the better model on this building:** test MAE 7.81 kW against 9.38 kW for the MLP, and a daily-peak error of 6.92 kW against 10.16 kW. The validation choice agreed with the test ranking.
- **The ensemble settled on 1.00 × LSTM + 0.00 × MLP.** Mixing in the MLP made validation error worse, so the fitted ensemble is the LSTM alone here. On the bundled synthetic demo building the same procedure chose 0.40 × LSTM + 0.60 × MLP, and the blend beat both models (test MAE 5.15 against 5.43 kW).
- **The 80 % range held 86.3 % of actual test readings** (80.0 % on validation), so it is slightly wider than needed: a safe error.
- **Error grows with lead time** from about 6.4 kW one hour ahead to 7.7 kW 24 hours ahead for the LSTM.

Test error by forecast lead time



Mean absolute error on the test period for each hour ahead. The ensemble line coincides with the LSTM line because its fitted weight is 1.00 × LSTM.



24-hour forecast after the last reading. Peak 96.1 kW at 21:00, lowest 71.4 kW at 04:00, 2,066 kWh over the day. Shaded: 80 % range. The ensemble (blue) lies under the LSTM line (orange); the MLP (green) is shown for comparison. 1 January 2018 is a public holiday, and the models have no holiday calendar.

5 Key design decisions

- **Time order is never broken.** Splits are by date, the scaler sees training data only, gaps are filled forward only, and a forecast window counts only if all its actual readings are real. This is what makes the accuracy figures believable.
- **Choose on validation, report on test.** Model selection and the ensemble weight use the validation period; the test period is touched once, for the final scores.
- **Two different models, then a fitted blend.** The LSTM and MLP make different mistakes. The blend weight is learned, not assumed, and can fall back to a single model when mixing does not help, as it did here.
- **Every forecast says how reliable it is.** Typical error, peak error and an 80 % range accompany each forecast, and the range's real coverage is measured.
- **One report format everywhere.** Command line, web app and assistant read the same files.
- **Clear errors instead of silent guesses.** Too little data, the wrong CSV or an out-of-date ensemble stop with an explanation of what to do.

6 Technology

Area	Tools
Language	Python 3.11
Models	TensorFlow / Keras (LSTM, MLP)
Data	pandas, NumPy; scikit-learn for scaling and TF-IDF document search
Charts	matplotlib (evaluation plots), Altair / Vega-Lite (web app)
Web app	Streamlit
Assistant	Anthropic Claude API (claude-opus-5-5), structured JSON answers; pypdf for PDF documents
Quality	82 unit and integration tests (unittest), including a headless run of the web app

7 Status and limitations

- Trained and evaluated on one real office building. Each new building needs its own training run, which takes a few minutes on a CPU.
- No holiday calendar: holidays are forecast like ordinary days of the same weekday.
- Future weather is not used; forecasts rely on conditions up to the forecast time.
- The Building Data Genome data has no occupancy or equipment status, so those features were not used here.
- The question-answering assistant runs from the terminal only and needs an Anthropic API key; each question is a paid call.

8 How to run it

```
pip install -r requirements.txt
streamlit run streamlit_app.py          # web app: http://localhost:8501

python -m src.train --model both --csv data\bdg2_hog_office_lizzie.csv
python -m src.evaluate --csv data\bdg2_hog_office_lizzie.csv
python -m src.predict --model best --csv data\bdg2_hog_office_lizzie.csv
```

9 Data credit

Building data: **Building Data Genome Project 2**, Miller, C. et al. (2020), *Scientific Data* 7, 368, github.com/buds-lab/building-data-genome-project-2, licensed CC BY-SA 4.0. Modified for this project: one building selected, site weather joined, relative humidity derived from dew point.